

Hierarchical Constraint-Aware Design-Space Exploration for Heterogeneous Distributed Cyber-Physical Systems

Nikita Tarasov^a, Roman Zinko^a

^a*Lviv Polytechnic National University, Lviv, Ukraine*

Abstract

Architecting a heterogeneous distributed cyber-physical system requires simultaneous decisions on the number and class of edge and gateway nodes, on where application stages are executed, on replication, and on communication links, under conflicting latency, energy, cost and network-volume objectives. The resulting design space grows combinatorially, and for distributed cyber-physical systems a single design point is expensive to evaluate, so exhaustive exploration is impractical while population-based search returns approximate fronts without guarantees. This paper presents HCA-DSE, a hierarchical constraint-aware design-space exploration method that discards structurally ill-formed, provably infeasible and provably dominated regions of the architecture space without fully evaluating each discarded candidate. Its central technical ingredient is a family of objective-specific admissible relaxations for a finite heterogeneous edge-cloud CPS model with *contention-dependent* objectives: the end-to-end latency of a distributed cyber-physical system depends on tier utilisation, which is not determined by any single design decision, so a sum of assigned decision costs is not by itself an admissible bound. We prove that the resulting pruning is safe and that the exact Pareto front is preserved, and we verify both the bound admissibility and the front equality exhaustively on reference spaces. Across four design-space scales and a scalability sweep up to 8.3×10^7 candidate architectures, the default method performs 16–63 full evaluations. It preserves the exhaustive analytical Pareto front on the reference spaces S1–S3; exactness on the largest spaces rests on the bound proof. Budget-matched NSGA-II attains median Pareto recall of 0.65–0.94 at 1000 actual evaluator calls. An evaluation-cost model anchored at a measured DES cost of 33.6 ms projects speed-ups up to 758×; this is not an end-to-end DES search measurement. We additionally isolate variable-order and queue-bound effects and compare against an exact finite-table SMT baseline. An independent containerised validation covers 36 architectures and 108 runs with the model frozen. For architecture pairs separated by more than 25% in predicted latency, agreement with measured mean-latency order is 88.6–91.3% within workloads (97.8% pooled).

Keywords: design space exploration, cyber-physical systems, edge computing, multi-objective optimisation, Pareto optimality, architecture exploration, design automation

1. Introduction

Distributed cyber-physical systems (dCPS) are increasingly deployed on a heterogeneous compute continuum that spans sensing devices, edge nodes, gateways and cloud infrastructure. Designing such a system is not a single decision but a joint one: how many edge nodes and gateways

Email addresses: `dev.nikita@outlook.com` (Nikita Tarasov), `roman.v.zinko@lpnu.ua` (Roman Zinko)

to deploy, which hardware class each of them should be, where each application stage is executed, how far the ingest stage is replicated, and which class of communication link connects the tiers. These decisions interact. Adding edge nodes reduces queueing delay but raises deployment cost and static energy; moving analytics to the cloud removes load from the edge tier but adds a wide-area transfer to every request; replication duplicates ingest execution and multiplies both compute load and synchronisation traffic.

Because the decisions are discrete and interacting, the number of candidate architectures grows as the product of the decision domains and quickly reaches 10^5 to 10^7 even for a modest set of design variables. At the same time, evaluating a single candidate is expensive: a recent industrial dCPS exploration methodology reports that “evaluating each design point takes under a minute” with a discrete-event simulator, and its authors explicitly leave the integration of a search strategy out of scope [1]. A survey of the field concludes that “efficient and scalable DSE technology for dCPS is more or less non-existing and constitutes a largely uncharted research area” and identifies the cost of a design-point evaluation as a defining difficulty of the distributed setting [2]. The combination of a large space and an expensive evaluator is precisely the regime in which the number of *full evaluations*, rather than the raw size of the space, determines whether design-space exploration (DSE) is practical at all.

Two families of approaches dominate. Exhaustive or model-based enumeration evaluates every candidate and therefore yields the exact Pareto set, but its cost scales with the size of the space. Population-based metaheuristics such as NSGA-II [3] operate under a fixed evaluation budget and return an approximation with no guarantee on which trade-offs have been missed; Section 6 measures its recall and hypervolume under precisely matched full-evaluator-call budgets. A third, smaller family performs *exact* multi-objective exploration by pruning partial designs, either symbolically with constraint solvers [4, 5] or through bound-based Pareto pruning in the broader optimisation literature [6, 7]. Assignment-monotone partial costs are one way to obtain admissible bounds, but exact bound-based search does not require all objectives to be assignment monotone. Consistent approximations have also been used to avoid expensive DSE evaluations while retaining exactness [8].

Here we study a concrete contention-dependent architecture model. Latency contains a waiting component governed jointly by node counts, resource classes, replication and placement. We derive objective-specific relaxations for this model and quantify their effect on exploration. The contribution is the model, bound construction and empirical analysis, rather than a new general principle of safe Pareto pruning.

Contributions.

1. A formal design-space model for heterogeneous edge–gateway–cloud CPS architectures (Section 3) with analytical models of latency including contention, energy, normalised deployment cost, network volume, and six classes of feasibility constraints.
2. **Admissible bounds for contention-dependent objectives** (Section 4). We decompose each objective into non-negative terms, bound every term independently over the still-reachable domains, and include the waiting term exactly once its determinants are fixed. In the default order this occurs at full assignment; an alternative-order experiment tests earlier activation. Lemmas 1 and 2 establish admissibility.
3. **HCA-DSE**, a hierarchical exploration method that combines structural well-formedness, feasibility pruning and bound-based dominance pruning with a final exact evaluation of the

survivors, together with proofs (Propositions 1–3, Theorem 1) that the exact Pareto front is preserved.

4. **An evaluation-cost analysis** that answers when such pruning is worth its own overhead, including a measured break-even evaluator cost and a crossover curve (Section 6).
5. **An empirical validation** on a containerised edge–gateway–cloud testbed with per-node link shaping: an independent round of 36 fixed architectures and 108 measured runs, reported both as prediction error and as the agreement between the architecture ordering produced by the model and the ordering observed in the measurements.
6. A reproducibility package in which every figure and every numeric table of this paper is generated from a checksummed canonical result set, and in which the correctness claims are backed by exhaustive machine-checked tests.

The remainder of the paper is organised as follows. Section 2 positions the work. Section 3 defines the design space and the evaluation model. Section 4 presents HCA-DSE and its correctness. Section 5 describes the experimental methodology, Section 6 the results. Section 7 discusses when the method pays off and when it does not, Section 8 the threats to validity, and Section 9 concludes.

2. Related work

2.1. Exact multi-objective design-space exploration

The closest methodological relatives of this work perform exact multi-objective exploration by reasoning about partial designs. Neubauer et al. [4] encode system-level synthesis in Answer Set Programming modulo Theories and attach theory propagators that check both constraints and Pareto dominance on *incomplete* assignments, returning conflict clauses that remove infeasible and dominated regions early. Their formulation targets heterogeneous multiprocessors with routing, optimising latency, area and energy, and their pruning is sound for problems whose objectives are assignment monotonous. In the broader optimisation literature, multi-objective A^* with admissible or consistent heuristics [9, 7] provides its theoretical foundation for bound-based pruning: a partial solution whose optimistic vector is dominated by a complete solution can be discarded without losing Pareto-optimal completions.

HCA-DSE belongs to this family, and we do not claim safe pruning as such as a contribution. The difference is the class of objectives. In a distributed CPS the dominant latency term is contention-dependent: it is a function of tier utilisation, which no single assignment determines. We therefore construct bounds that are admissible without assignment monotonicity, by combining a term-wise relaxation with an explicit decision ordering that makes the contention term exactly computable as soon as its determinants are fixed (Section 4). Avoiding expensive evaluations is also an explicit aim of consistent-approximation DSE [8], which combines safe approximations with exact assessment of promising designs. We therefore do not claim evaluator avoidance as a new principle. Our study specialises bounds to this CPS model and measures the evaluation-cost trade-off. The term “Pareto pruning” also refers to selecting a representative subset of an already computed front [6]; that operation differs from our exact search-space pruning.

2.2. Design-space exploration for distributed cyber-physical systems

Herget et al. [2] survey DSE for dCPS and identify the two premises of this paper: design spaces that are far larger than in classical MPSoC exploration, and design-point evaluations that are considerably more expensive. CompDSE [1] uses Y-chart modelling [10] and derives workload,

platform and mapping models from execution traces of an industrial lithography machine, evaluating design points with OMNeT++ [11]. It is an evaluator, not a search strategy: the authors report a manual exploration of three variations and state that integrating search algorithms is out of scope. The two approaches are therefore complementary — CompDSE answers how a design point should be evaluated, HCA-DSE answers which design points need not be evaluated at all — and the per-design-point cost reported there falls in the regime where our crossover analysis predicts speed-ups of two orders of magnitude.

ContrArc [12] explores CPS architectures with a mixed-integer program combined with contract refinement checking, pruning through subgraph isomorphism. It minimises a single objective (cost) subject to requirements rather than constructing a Pareto front, and its pruning is constraint-based rather than bound-based.

Simulation-based CPS exploration also includes the cyber/control co-design approach of Amir and Givargis [13]. Automated workload derivation for industrial dCPS [14] addresses the input-model construction problem. Edge CNN exploration [15] and its robustness extensions [16] illustrate application-specific architecture trade-offs at the edge. These studies motivate broader workloads and reliability models; they do not supply the objective bounds proved here.

2.3. *Exact allocation by integer programming*

A second exact family formulates allocation as an integer program and solves it to optimality. Kouloumpris et al. [17] allocate the tasks of an application over an edge/hub/cloud triple with binary integer linear programming, minimising latency or energy under computation, communication, memory and energy constraints, and validate on a UAV inspection use case with synthetic benchmarks of up to 1000 tasks. Their follow-up [18] extends the formulation to a reliability/latency problem solved exactly and offline for specified objective weights. This certifies a weighted optimum, not enumeration of the entire discrete Pareto front.

These works are exact in a different sense than the present paper, and the distinction matters for the choice of method. They solve one *allocation* instance over a fixed infrastructure with a model that is linear in the decision variables, whereas we explore an *architecture* space in which the number and class of nodes, the link class and the replication factor are themselves decisions, and in which the latency model contains a non-linear contention term. A compact mixed-integer linear formulation would require an encoding of the queueing term. Piecewise-linear approximations can change the objective, but finite-domain table or disjunctive encodings can in principle retain the exact values at the cost of model size. Nonlinearity therefore does not rule out an exact solver baseline. Section 6.7 measures a finite-table SMT baseline, including table construction; a compact symbolic encoding remains outside the comparison.

2.4. *Design-time placement in the edge–cloud continuum*

SPACE4AI-D [19] is the closest work in terms of the problem setting: design-time selection of resources and component placement across edge, cloud and function-as-a-service layers. The problem is formulated as a mixed-integer non-linear program with a single objective (deployment cost) under response-time constraints, and solved with randomised greedy, local search, tabu search, simulated annealing and genetic algorithms; no exactness guarantee is provided. Its validation on a real deployment reports deviations of 12% in cost and 20% in execution time, which is a useful reference point for the accuracy reported in Section 6.

Recent Journal of Systems Architecture studies provide complementary context. Casini et al. [20] combine design-time allocation and timing analysis with runtime monitoring for real-time

Table 1: Positioning with respect to the closest prior work. “Partial-design pruning” denotes discarding incomplete designs before they are completed and evaluated; “contention-aware” denotes objectives that depend on resource utilisation rather than being additive over decisions. “Weighted” denotes exact optimisation for specified weights, without full-front enumeration.

Work	Year	System class	Design variables	Obj.	Exact front	Partial-design	Cont.-aware	Exp. eval.	Real meas.
Blickle et al. [30]	1998	embedded synthesis	allocation, binding	2	no	no	no	no	no
Haubelt & Teich [31]	2003	SoC	component selection	n	partly	front comp.	no	no	no
Lukasiewicz et al. [32]	2008	embedded synthesis	allocation, binding, routing	n	no	SAT-based	no	no	no
Neubauer et al. [4]	2018	heterog. MPSoC	allocation, binding, routing	3	yes	symbolic	no	no	no
Herget et al. [2]	2022	distributed CPS	survey	—	—	—	—	—	—
Kouloumpris et al. [17]	2024	edge/hub/cloud	task allocation	1	n/a	solver	no	no	UAV case
Xiao et al. [12]	2024	CPS architectures	component selection	1	—	isomorphism	no	no	no
Sedghani et al. [19]	2024	computing continuum	resources, placement	1	no	no	partly	no	yes
CompDSE [1]	2025	industrial dCPS	cores, hosts, frequency	perf.	—	none	yes	—	traces
Kouloumpris et al. [18]	2026	edge/hub/cloud	task allocation	2	weighted	solver	no	no	case study
NSGA-II family [3]	—	edge–cloud	varies	2–4	no	no	partly	no	rarely
HCA-DSE (this work)	2026	heterog. edge–cloud CPS	counts, classes, links, placement, replication	4	yes	bounds	yes	yes	yes

edge applications. Workflow scheduling in MEC-supported CPS [21] addresses constrained offloading and resource allocation with a metaheuristic. Dependency-aware microservice offloading has also been studied on an ICN edge testbed [22]. More recent work examines energy-efficient cloud–edge inference co-optimisation [23], distributed replica allocation and load balancing [24], and predictive energy-aware continuum orchestration [25]. These are relevant deployment and optimisation settings, but their application models and dynamic decisions differ from the fixed, finite architecture space for which we certify an analytical Pareto front.

2.5. Metaheuristic exploration of edge–cloud deployments

A large body of work applies evolutionary and swarm metaheuristics to task allocation, offloading and service placement in edge–cloud systems, typically optimising combinations of latency, energy and cost. These methods are approximate by construction and their quality is a function of the evaluation budget. We use NSGA-II [3] as implemented in pymoo [26] as the budget-matched representative of this family, and report hypervolume [27], IGD^+ [28] and Pareto recall against the exhaustive ground truth; the choice of indicators follows the guidance of Audet et al. [29]. Our conclusion is not that such methods are inferior in general, but that when admissible bounds are available exactness is achievable at a small fraction of their evaluation budget.

Table 1 summarises the comparison.

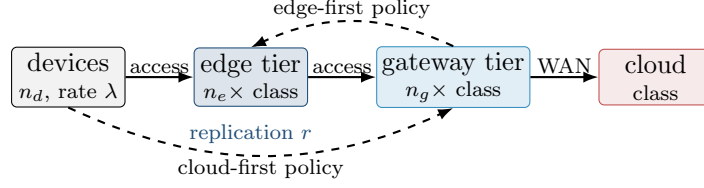


Figure 1: Architecture template. Solid arrows show the default request path; dashed arrows show the alternative stage placements selected by the placement policy.

Table 2: Resource classes. Compute classes: capacity C [MIPS], memory M [MB], idle and peak power [W], cost [units]. Link classes: bandwidth B , propagation delay τ , transfer energy e , cost.

class	C / B	M / τ	P^{idle} / e	P^{peak}	cost
<i>compute node classes</i>					
edge-low	2000	2048	4	12	1
edge-med	6000	8192	8	30	2.4
edge-high	15000	16384	15	65	5.2
gw-low	4000	4096	6	20	1.8
gw-high	12000	16384	12	48	4
cloud-med	40000	131072	0	180	6
cloud-high	100000	262144	0	380	12
<i>link classes (name, B [Mb/s], τ [ms], e [J/MB], cost)</i>					
constrained	10	4	0.3	0.2	–
standard	100	2	0.12	0.6	–
fast	1000	1	0.05	1.4	–

3. System and design-space model

3.1. Architecture template

We consider the four-tier template of Fig. 1. A population of devices generates requests; each request traverses three application stages. The *pre-processing* stage τ_1 runs either on the edge tier or on the gateway tier, the *aggregation* stage τ_2 always runs on the gateway tier, and the *analytics* stage τ_3 runs either on the edge tier or in the cloud. Which of these variants is instantiated is a design decision (the placement policy).

A compute node class is a tuple $(C, M, P^{\text{idle}}, P^{\text{peak}}, c)$ of compute capacity [MIPS], memory [MB], idle and peak power [W] and normalised cost per node. A link class is a tuple $(B, \tau^{\text{prop}}, e, c)$ of bandwidth [Mb/s], propagation delay [s], transfer energy [J/MB] and cost per attached node. The gateway-to-cloud connection is fixed infrastructure ($B = 200$ Mb/s, $\tau^{\text{prop}} = 20$ ms) and is not a design variable. Table 2 lists the concrete classes used in the experiments.

3.2. Workload

A workload is $(n_d, \lambda, \{\tau_i\}_{i=1}^3, D, s^{\text{sync}})$ where n_d devices each emit requests at rate λ , giving a system arrival rate $\Lambda = n_d \lambda$; stage $\tau_i = (w_i, m_i, s_i)$ has computational work w_i [MI], resident memory m_i [MB] and inbound payload s_i [MB]; D is the end-to-end deadline and s^{sync} the per-request replica synchronisation payload. Three workload classes are used throughout (Table 3); they are qualitatively different rather than numerically rescaled versions of one another: telemetry is high-rate and small-payload, real-time control is deadline-dominated, and data-intensive sensing is payload- and compute-dominated.

Table 3: Workload classes. Stage values are given as $\tau_1/\tau_2/\tau_3$.

workload	n_d	λ	Λ	w_i [MI]	s_i [kB]	D [ms]	$C^{\max}$
telemetry	200	2	400	8/4/20	20/10/10	120	60
control	60	10	600	5/3/9	5/4/3	30	60
sensing	40	1.5	60	120/30/260	1200/350/300	900	90

Table 4: Design variables and domain sizes of the four design-space scales.

variable	meaning	S1	S2	S3	S4
n_e	number of edge nodes	3	6	24	80
$edge_cls$	edge node class	3	3	3	3
n_g	number of gateways	2	3	5	8
gw_cls	gateway class	2	2	2	2
rep	replication factor of stage 1	2	3	4	5
$policy$	placement policy	3	3	3	3
$link_cls$	access link class	3	3	3	3
$cloud_cls$	cloud instance class	2	2	2	2
<i>design-space size</i> $ \mathcal{X} $		1 296	5 832	51 840	345 600

3.3. Decision vector and design space

An architecture is described by the vector

$$x = (n_e, \kappa_e, n_g, \kappa_g, \ell, \kappa_c, r, \pi), \quad (1)$$

with n_e, n_g the number of edge nodes and gateways, $\kappa_e, \kappa_g, \kappa_c$ the edge, gateway and cloud classes, ℓ the access link class, r the replication factor of stage τ_1 and π the placement policy. The design space is the Cartesian product $\mathcal{X} = D_1 \times \dots \times D_k$ with $k = 8$, and $|\mathcal{X}| = \prod_i |D_i|$. Table 4 lists the domains of the four scales used in the experiments.

3.4. Evaluation model

Let $t(i)$ denote the tier executing stage i under policy π , N_t the number of nodes of tier t and C_t the per-node capacity of its class. The offered load and utilisation of a tier are

$$\Theta_t = \sum_{i: t(i)=t} \Lambda w_i \mu_i, \quad \rho_t = \frac{\Theta_t}{N_t C_t}, \quad \mu_i = \begin{cases} r & i = 1 \\ 1 & \text{otherwise,} \end{cases} \quad (2)$$

that is, replication multiplies the load of the ingest stage but not of the others.

Latency. We use a stage-wise M/D/1-inspired waiting approximation [33, 34]: each stage retains its per-node service time $w_i/C_{t(i)}$, while the waiting multiplier uses the average tier utilisation ρ_t based on aggregate capacity $N_t C_t$. This is not an exact M/D/1 queue with service capacity $N_t C_t$: such a queue would also divide the service time by N_t . Nor is it an exact multi-server model. Stages sharing a tier have different service demands, and routing, replication and successive visits can violate the Poisson-arrival assumption. The expression is therefore a tractable surrogate whose prediction error must be assessed empirically; it is not a proven lower bound on deployment latency. The simulator and testbed implement per-node servers and quantify this gap. With $\mathcal{H}(x)$ the set

of communication hops implied by the policy and the replication factor,

$$L(x) = \underbrace{\sum_{i=1}^3 \frac{w_i}{C_{t(i)}}}_{\text{service}} + \underbrace{\sum_{i=1}^3 \frac{w_i}{C_{t(i)}} \cdot \frac{\rho_{t(i)}}{2(1-\rho_{t(i)})}}_{\text{waiting}} + \underbrace{\sum_{(s,\ell) \in \mathcal{H}(x)} \left(\frac{8s}{B_\ell} + \tau_\ell^{\text{prop}} \right)}_{\text{communication}}. \quad (3)$$

The middle term is what makes the objective *contention-dependent*: it is a function of ρ_t , which by (2) depends jointly on n_t , the class of the tier, the replication factor and the policy. If $\rho_t \geq 1$ for any used tier the architecture is discarded as unstable.

Energy, cost and network volume. Over an accounting horizon T ,

$$E(x) = \sum_{t \in \{e,g\}} N_t \left(P_t^{\text{idle}} + \rho_t (P_t^{\text{peak}} - P_t^{\text{idle}}) \right) T + \rho_c P_c^{\text{peak}} T + \sum_{(s,\ell) \in \mathcal{H}(x)} s \Lambda e_\ell T, \quad (4)$$

$$C(x) = n_e (c_{\kappa_e} + c_\ell) + n_g (c_{\kappa_g} + c_\ell) + c_{\kappa_c}, \quad (5)$$

$$D(x) = \Lambda \sum_{(s,\ell) \in \mathcal{H}(x)} s. \quad (6)$$

The objective vector is $F(x) = (L(x), E(x), C(x), D(x))$, all components minimised. Energy is the total consumption over the fixed accounting horizon $T = 1$ h and is reported in kWh; the cost coefficients of Table 2 are synthetic normalised values encoding the relative price tiers of the resource classes, and no monetary interpretation is claimed. Neither the energy nor the cost model is validated against measurements in this paper: they are used for comparative exploration only.

Feasibility. An architecture is feasible when all of the following hold: $\rho_t \leq \rho^{\text{max}}$ for every used tier; the resident memory of the stages mapped to a tier fits in the memory of its node class; the aggregate access-link traffic does not exceed the total bandwidth of the attached nodes; $L(x) \leq D$; $C(x) \leq C^{\text{max}}$; and $E(x) \leq E^{\text{max}}$. We write $\mathcal{F}$ for the set of feasible architectures.

4. HCA-DSE

4.1. Partial architectures

The exploration traverses a prefix tree over the decision vector (1) under a fixed variable order $v_1, \dots, v_k$.

Definition 1 (Partial architecture). A partial architecture of depth d is a prefix of assignments $p = (a_1, \dots, a_d)$, $a_i \in D_i$. Its completions are $\mathcal{C}(p) = \{x \in \mathcal{X} : x_i = a_i, i \leq d\}$, with $|\mathcal{C}(p)| = \prod_{i>d} |D_i|$.

Let $\sigma : \mathcal{X} \rightarrow \{0, 1\}$ be the well-formedness predicate of the template: the number of gateways does not exceed the number of edge nodes, the replication factor does not exceed the number of edge nodes, and replication is undefined for policies that do not place the ingest stage on the edge tier. The working space is $\mathcal{X}_\sigma = \{x : \sigma(x) = 1\}$ and all statements below refer to $\mathcal{F}_\sigma = \mathcal{F} \cap \mathcal{X}_\sigma$; the exhaustive baseline against which we compare uses the same predicate. The Pareto front is

$$\mathcal{P}^* = \{F(x) : x \in \mathcal{F}_\sigma, \nexists y \in \mathcal{F}_\sigma : F(y) \preceq F(x)\}, \quad (7)$$

where $u \preceq v$ denotes $u_i \leq v_i$ for all i and $u_j < v_j$ for at least one j .

4.2. Admissible bounds

Definition 2 (Admissibility). LB_i is admissible for objective f_i if $LB_i(p) \leq f_i(x)$ for every partial architecture p and every $x \in \mathcal{C}(p)$.

The partial-cost construction discussed by Neubauer et al. assumes that objectives are *assignment monotonous*, i.e. that every assignment can only increase the objective, so that the partial cost is itself a bound [4]. The waiting term of (3) violates this assumption: it depends on ρ_t , which by (2) is jointly determined by n_t , the class of the tier, r and π , and which *decreases* when nodes are added. We therefore construct the bounds in two steps.

Lemma 1 (Term-wise relaxation). Let $f_i(x) = \sum_{t \in T_i} \varphi_t(x)$ with $\varphi_t \geq 0$, and let

$$\underline{\varphi}_t(p) = \min_{x \in \mathcal{C}(p)} \varphi_t(x).$$

Then $LB_i(p) = \sum_{t \in T_i} \underline{\varphi}_t(p)$ is admissible.

Proof. For every $x \in \mathcal{C}(p)$ and every t , $\underline{\varphi}_t(p) \leq \varphi_t(x)$ by definition of the minimum. Summing over $t \in T_i$ yields $LB_i(p) \leq f_i(x)$. $\square$

The minima are taken *independently per term*, so the combination of values that realises them need not correspond to any single completion; the bound is therefore optimistic but valid. The implementation uses domain extrema for service, communication, static energy and cost, and minima of class-specific power-to-capacity ratios for dynamic energy. Categorical policies are enumerated over their small reachable set; they have no meaningful monotone endpoint order. Independent extrema can relax a term further than its exact minimum, which remains safe. The queueing contribution is bounded by zero until its determinants are fixed. Domain aggregates are precomputed and repeated class-dependent terms cached; cache misses can still require scans of reachable classes or policies.

Lemma 2 (Exactness on determined terms). Let $\text{supp}(\varphi_t)$ be the set of variables φ_t depends on. If $\text{supp}(\varphi_t) \subseteq \{v_1, \dots, v_d\}$ then φ_t is constant on $\mathcal{C}(p)$, and substituting its exact value for $\underline{\varphi}_t(p)$ preserves admissibility while tightening the bound.

Proof. Constancy is immediate, and a constant equals its own minimum over $\mathcal{C}(p)$, so Lemma 1 applies unchanged. $\square$

Remark 1 (Why the variable order matters). Once all determinants of a tier’s waiting contribution are assigned, that contribution is constant on $\mathcal{C}(p)$ and can enter the bound exactly by Lemma 2; before then, zero is admissible for stable completions. If a determined used tier has $\rho_t \geq 1$, its latency is infinite. The default order O1 is $(n_e, \kappa_e, n_g, \kappa_g, \ell, \kappa_c, r, \pi)$. Policy is assigned last, so its exact queueing contribution becomes available only at depth eight. We separately test O2, $(n_e, \kappa_e, n_g, \kappa_g, r, \pi, \ell, \kappa_c)$, which can fix edge and gateway queueing determinants on incomplete designs. Section 6.6 separates the benefit of this term from the effect of changing traversal order.

4.3. Pruning rules

Proposition 1 (Safe feasibility pruning). Let LB_{g_j} be admissible for constraint function g_j . If $LB_{g_j}(p) > g_j^{\max}$ then $\mathcal{C}(p) \cap \mathcal{F} = \emptyset$.

Proof. For any $x \in \mathcal{C}(p)$, $g_j(x) \geq LB_{g_j}(p) > g_j^{\max}$, hence $x \notin \mathcal{F}$. $\square$

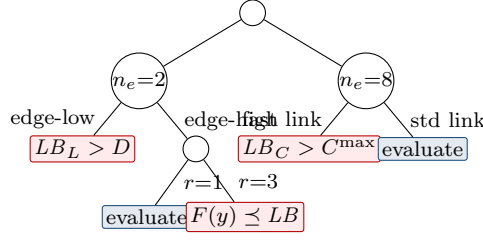


Figure 2: Pruning on the prefix tree. Sub-trees are discarded when the optimistic bound already violates a constraint (Proposition 1) or is dominated by an evaluated feasible design (Proposition 2); only surviving leaves reach the expensive evaluator.

Proposition 2 (Safe dominance pruning). *Let $y \in \mathcal{F}_\sigma$ be a fully evaluated feasible architecture with $F(y) \preceq LB(p) = (LB_1(p), \dots, LB_n(p))$. Then $F(x) \notin \mathcal{P}^*$ for every $x \in \mathcal{C}(p)$.*

Proof. Let $x \in \mathcal{C}(p)$. By admissibility $f_i(x) \geq LB_i(p) \geq F_i(y)$ for all i , and by assumption there is a j with $F_j(y) < LB_j(p) \leq f_j(x)$. Hence y strictly dominates x and $F(x)$ is not on the Pareto front. $\square$

Proposition 3 (Closure of the well-formedness predicate). *Each invariant of σ depends only on assigned variables and, under the chosen order, all variables of an invariant are assigned before it is first checked. Since assignments are never revised, $\neg\sigma(p)$ implies $\sigma(x) = 0$ for every $x \in \mathcal{C}(p)$, i.e. $\mathcal{C}(p) \cap \mathcal{X}_\sigma = \emptyset$.*

Remark 2. Structural invariants define what counts as an architecture rather than following from the constraints of the problem: the evaluator can formally compute objectives for, say, a replicated deployment under a policy that does not place the ingest stage on the edge tier, although such a configuration is meaningless. Structural pruning is therefore part of the definition of the search space, and the exhaustive baseline applies the same predicate. This distinction was found by the test described in Section 5, which initially — and incorrectly — verified structural pruning as a feasibility argument.

4.4. Algorithm

Algorithm 1 traverses the prefix tree depth-first. At every node the three rules are applied in increasing order of cost: structural validity, then feasibility bounds, then dominance against an archive of non-dominated evaluated points. A leaf that survives is evaluated in full; feasible leaves update the archive. Fig. 2 illustrates the two bound-based rules.

4.5. Correctness

Theorem 1 (Pareto-front preservation). *If all bounds are admissible, pruning is performed only by Propositions 1–3, and the archive contains only feasible fully evaluated architectures, then the set of non-dominated vectors returned by Algorithm 1 equals the true Pareto front: $\mathcal{P}^{\text{HCA}} = \mathcal{P}^*$.*

Proof. ($\subseteq$) Every returned vector is feasible. If a returned vector were dominated by any feasible vector, finiteness of $\mathcal{X}$ implies that it is dominated by a Pareto-optimal vector. The following argument shows that every Pareto-optimal vector is evaluated; it would therefore remove the dominated vector during final filtering, a contradiction.

Algorithm 1 HCA-DSE

Require: design space $\mathcal{X}$ with order $v_1..v_k$, workload w , evaluator F

Ensure: Pareto front $\mathcal{P}$

```
1:  $A \leftarrow \emptyset$  ▷ archive of non-dominated evaluated vectors
2:  $S \leftarrow \emptyset$  ▷ evaluated feasible architectures
3: procedure EXPLORE( $p, d$ )
4:   if  $d = k$  then
5:      $(F(p), \text{feasible}) \leftarrow \text{EVALUATE}(p, w)$  ▷ the expensive step
6:     if  $\text{feasible}$  then  $S \leftarrow S \cup \{p\}$ ; ARCHIVEINSERT( $A, F(p)$ )
7:   end if
8:   return
9: end if
10: for all  $a \in D_{d+1}$  do
11:    $q \leftarrow p \cup \{v_{d+1} \mapsto a\}$ 
12:   if  $\neg \sigma(q)$  then continue ▷ Prop. 3
13: end if
14:    $LB \leftarrow \text{BOUNDS}(q)$  ▷ Lemmas 1, 2
15:   if  $\exists j : LB_{g_j}(q) > g_j^{\max}$  then continue ▷ Prop. 1
16: end if
17:   if  $\exists f \in A : f \preceq LB$  then continue ▷ Prop. 2
18: end if
19:   EXPLORE( $q, d + 1$ )
20: end for
21: end procedure
22: EXPLORE( $\emptyset, 0$ )
23: return non-dominated subset of  $\{F(x) : x \in S\}$ 
```

($\supseteq$) Let $u \in \mathcal{P}^*$ and let $x \in \mathcal{F}_\sigma$ with $F(x) = u$. The traversal is a prefix tree, so x is reached through the unique chain $p_0 \subset p_1 \subset \dots \subset p_k = x$. Suppose some p_d were pruned. By Proposition 3 this implies $\sigma(x) = 0$ and by Proposition 1 $x \notin \mathcal{F}$, both contradicting $x \in \mathcal{F}_\sigma$; by Proposition 2 it implies the existence of a feasible y strictly dominating x , contradicting $u \in \mathcal{P}^*$. Hence no p_d is pruned, x is evaluated, and u survives the final filtering. $\square$

Remark 3 (Duplicates). The theorem is stated in objective space. The strict dominance rule does not prune a design merely because its objective vector equals an archived vector. A separate duplicate-removal step may retain one representative per vector without changing the front. All experiments therefore compare sets of objective vectors.

Remark 4 (Order independence). Proposition 2 holds for any archive of evaluated feasible architectures, so the traversal order affects pruning efficiency but never correctness.

4.6. Complexity

Let C_E be the cost of a full evaluation, C_P the cost of a bound computation, N_B the number of checked prefixes, N_E the number of full evaluations and $|A|$ the maximum archive size. The search performs

$$T_{\text{HCA}} = O(N_B(C_P + n|A|) + N_EC_E). \quad (8)$$

For measured cost projections we use $T_H = O_H + N_E C_E$ and $T_X = O_X + N_X C_E$, where N_X counts structurally valid designs evaluated by exhaustive search, and O_H, O_X are the respective measured non-evaluation overheads. Provided $N_X > N_E$, the break-even condition is

$$C_E > \frac{O_H - O_X}{N_X - N_E}. \quad (9)$$

A negative threshold means HCA-DSE is already favoured at zero evaluator cost in this linear model. Using $|\mathcal{X}|$ in place of N_X would incorrectly charge exhaustive search for structurally rejected designs. Expanded partial states N , reported separately, exclude branches rejected before recursion; N_B counts their bound checks as well. The Cartesian product is never materialised by HCA-DSE.

5. Experimental methodology

5.1. Research questions

RQ1 By how much does HCA-DSE reduce the number of *full design-point evaluations*, and how is that reduction distributed over the pruning layers?

RQ2 Is the exact Pareto front preserved with respect to exhaustive enumeration?

RQ3 At which evaluator cost does the pruning overhead pay for itself?

RQ4 How does HCA-DSE compare with budget-matched stochastic exploration?

RQ5 How do the reduction and the runtime scale with the size of the design space?

RQ6 Do the architecture rankings produced by the analytical model transfer to a real containerised edge-cloud deployment?

5.2. Design of the experiments

Four design-space scales (S1–S4, Table 4) are combined with the three workload classes of Table 3. Exhaustive ground truth is computed for S1–S3; S4 and the scalability study are explored by HCA-DSE only. Search counts are deterministic for a fixed traversal order. Stochastic baselines use 20 independent seeds; DES and deployment measurements have separately specified repetitions.

Baselines. Exhaustive enumeration provides the ground-truth Pareto front. Random search and NSGA-II [3] (pymoo [26], population 40, SBX crossover, polynomial mutation, integer repair) are run under matched evaluation budgets of 100, 250, 500 and 1000 full evaluations, 20 seeds each. All methods use exactly the same evaluator and feasibility rules. A proposal rejected structurally consumes no full-evaluation budget. Every invocation on a structurally valid design consumes one unit, including invocations returning infeasibility and repeated designs. The stopping condition is exactly $N_E = B$, also for a partial final NSGA-II generation. We persist proposal counts, structural rejections, feasibility rejections and full evaluator calls; runtime includes proposal generation and rejection. Duplicate handling remains method-specific: random sampling is with replacement, while NSGA-II eliminates duplicates within its population.

Metrics. Pareto recall and precision are computed on sets of objective vectors against the exhaustive front, identifying objective vectors at nine decimal places for these quality metrics. Exactness experiments additionally compare unrounded binary64 objective-vector sets. Hypervolume [27] is normalised by the hypervolume of the exhaustive front (HV ratio); IGD⁺ [28] uses the exhaustive front as reference. We additionally report the number of full evaluations N_E , the number of expanded partial states N (recursive visits), and wall-clock time. Stochastic baselines are compared with the Mann–Whitney U test [35] and Cliff’s δ [36]. The Mann–Whitney alternative is that NSGA-II has larger HV values than random search; positive δ favours NSGA-II. Reported p -values are unadjusted and comparisons are exploratory.

Ablation. Four variants isolate the contribution of each layer: structural only; structural + feasibility bounds; structural + dominance bounds; and the full method. A separate 2×2 experiment crosses orders O1/O2 (Remark 1) with the queueing bound enabled/disabled on S1–S3. We record expanded states, bound calls, full evaluations, and exact front equality. An untimed instrumented pass counts incomplete prefixes with a positive queueing contribution and those pruned only because of that contribution against the same current archive. Runtime is the median of three uninstrumented runs.

Finite-table exact baseline. On S1/S2 we also use Z3 [37] to enumerate non-dominated points of an exact finite relation. Every structurally valid design is first evaluated; feasible objective vectors are encoded by per-objective integer ranks preserving the strict ordering of the stored binary64 values, with no linearisation or rounding. Repeated dominance queries find a non-dominated point and block its dominated region until UNSAT. Reported time includes all evaluator calls, table construction, encoding and solver queries. This tests an exact table encoding, not a compact symbolic model capable of avoiding those calls.

5.3. Two-level evaluation and the cost model

The analytical evaluator of Section 3 (level A) is used for pruning, ground truth and all exactness claims. A discrete-event simulator (level B) provides a higher-fidelity, and therefore more expensive, evaluator: multi-server queues with a finite number of servers per tier, explicit link contention, deterministic service times and Poisson arrivals. Replicated first-stage jobs fork and join; arrivals after propagation are scheduled before claiming a server. The cloud and WAN each have one server. We time 40 feasible S1/telemetry designs, each with 3000 requests and 400 warm-up requests, and use their mean cost (33.6 ms) as a hardware- and sample-specific anchor. Search overheads in the projection are medians of three runs. Model validation uses 40 distinct S2 designs per workload and five DES seeds, reporting the median simulated mean latency. DES is a separate queueing model; its front is not covered by the analytical exactness proof.

5.4. Testbed

The physical validation runs the architecture template as containers (Fig. 3). Each edge node, gateway and cloud instance is a separate container limited to one CPU, so that a node behaves as the single server assumed by the model. Services are implemented in Go and share one binary: a request carries a plan listing the remaining stages, and each node consumes CPU work equal to w_i/C_t before forwarding the remainder together with the real payload to the next hop. Links are shaped per node with `tc netem` [38] using the delay and rate of the link class of the design under test; the gateway-to-cloud hop uses the WAN profile. Replication is emulated faithfully: r

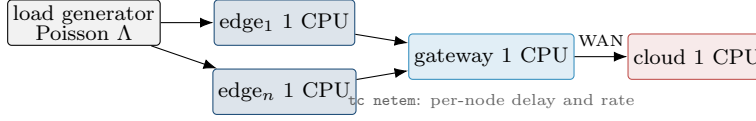


Figure 3: Containerised testbed. Every tier node is a separate one-CPU container; access links carry the delay and bandwidth of the link class of the design, the cloud hop carries the WAN profile.

replicas execute stage τ_1 in parallel and carry the synchronisation payload, while the remainder of the chain is executed once. A load generator produces Poisson arrivals at rate Λ , 900 requests per run of which the first 150 are discarded as warm-up, and reports the latency percentiles.

Thirty-six architectures were selected across the three workloads — Pareto-optimal, dominated and near-boundary designs — restricted to those that fit on a single machine (≤ 4 edge nodes, ≤ 2 gateways, one cloud instance). Each was executed three times, giving 108 measured runs in an independent second round. The configuration list and analytical model were frozen before this round; no parameters were fitted to its results. Network-shaping failures abort a run. We retain all repetitions, including unstable ones. The primary mean-latency comparison uses the median of three measured run means; we additionally retain the historical p_{50} comparison, which targets a different latency statistic and is not a direct validation of an expected-value approximation.

Because the analytical model is deliberately coarse, we report both the prediction error (MAPE, median absolute error) and, more importantly for exploration, the *ordering* agreement: Spearman’s ρ , Kendall’s τ , and the share of architecture pairs whose measured order agrees with the model order among the pairs that the model separates by more than 10% and 25%.

5.5. Reproducibility

The implementation is in Python 3 with NumPy, SciPy, pymoo and Z3; the testbed services are in Go and run under Docker Compose. All figures and all numeric tables of this paper are generated from the canonical **v1.1-submission-results** data; narrative numerical summaries are generated from the same files. The release records a clean source export, locked package versions, execution logs and SHA-256 hashes of sources and results. Archived CSVs are excluded from manuscript generation. A fresh interpreter rechecks search counters, budget identities, front equality and statistical summaries. Wall times and deployment measurements are not expected to reproduce bit for bit. Correctness is machine-checked at three levels: (i) bound admissibility is verified *exhaustively* on S1, i.e. for every prefix and every completion of that prefix across the three workloads; (ii) for every pruned prefix, all of its completions are enumerated and the claim of the corresponding proposition is verified directly, on S1 and S2; and (iii) the equality $\mathcal{P}^{\text{HCA}} = \mathcal{P}^*$ is re-checked on S1–S3 for every pruning variant as a regression test.

6. Results

6.1. RQ1: most architectures are discarded before evaluation

The exploration funnel is reported in Table 5 and Fig. 4. On S3 (51 840 candidates), HCA-DSE performs 43–56 full evaluations, removing 99.89–99.92% of the raw space before evaluation. On S4 (345 600 candidates), it performs 52–62 evaluations, a reduction of 99.98–99.98%. Raw-space reductions include structural rejection; Table 5 separately reports the smaller number of actual exhaustive evaluator calls.

Table 5: Exploration funnel. The reduction column refers to *full design-point evaluations* with respect to the raw design space, not to runtime; recall is against exhaustive enumeration where the latter is tractable.

scale	workload	$ \mathcal{X} $	structural pruned	feasibility pruned	dominance pruned	full evals	feasible	$ \mathcal{P} $	full-eval reduction [%]	exhaustive evals	recall
S1	telemetry	1 296	216	208	838	34	27	17	97.38	1 080	1.0
S1	control	1 296	216	226	826	28	25	18	97.84	1 080	1.0
S1	sensing	1 296	216	530	534	16	11	11	98.77	1 080	1.0
S2	telemetry	5 832	1 860	965	2 957	50	42	27	99.14	3 888	1.0
S2	control	5 832	1 860	1 017	2 916	39	36	25	99.33	3 888	1.0
S2	sensing	5 832	1 944	1 534	2 328	26	19	16	99.55	3 888	1.0
S3	telemetry	51 840	12 240	21 957	17 587	56	48	31	99.89	34 560	1.0
S3	control	51 840	12 240	21 924	17 633	43	40	27	99.92	34 560	1.0
S3	sensing	51 840	14 634	16 230	20 933	43	36	27	99.92	34 560	1.0
S4	telemetry	345 600	33 924	276 742	34 872	62	54	32	99.98	—	—
S4	control	345 600	33 924	276 811	34 812	53	50	29	99.98	—	—
S4	sensing	345 600	41 250	230 523	73 775	52	45	28	99.98	—	—

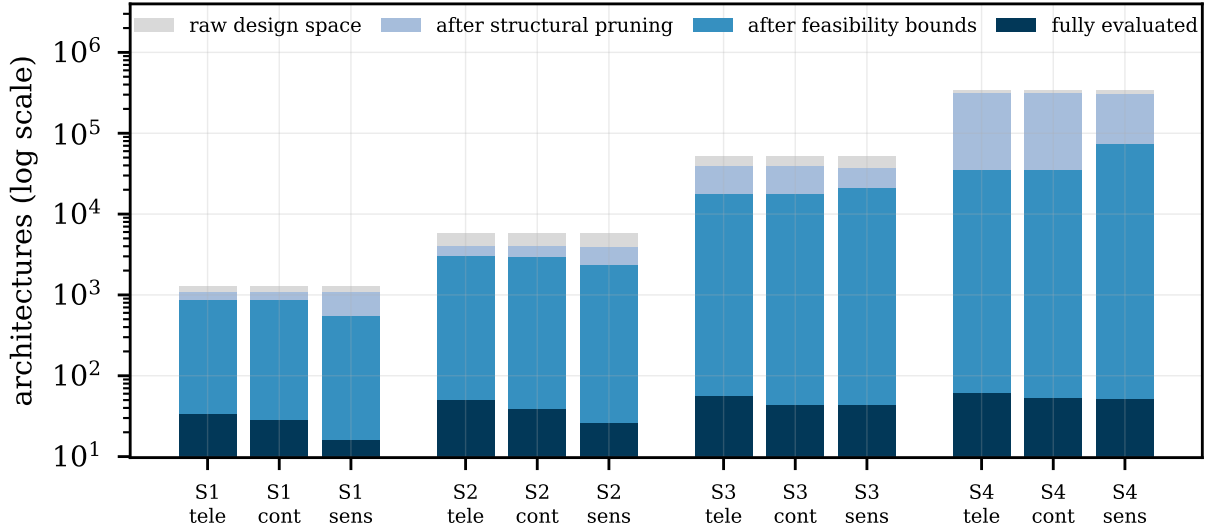


Figure 4: Design-space funnel across scales and workloads (logarithmic scale).

6.2. RQ2: the exact front is preserved

For every scale for which exhaustive enumeration is tractable (S1–S3) and for every workload, the set of objective vectors returned by HCA-DSE is identical to the exhaustive Pareto front: recall and precision are both 1.0. The same holds for each ablated variant, confirming that no individual pruning layer is unsafe. Bound admissibility was verified exhaustively on S1 — all prefixes against all completions — without a single violation, and for every pruned prefix on S1 and S2 all completions were enumerated and found consistent with the claim of the corresponding proposition. Fig. 5 shows the resulting front for S3/telemetry in two projections.

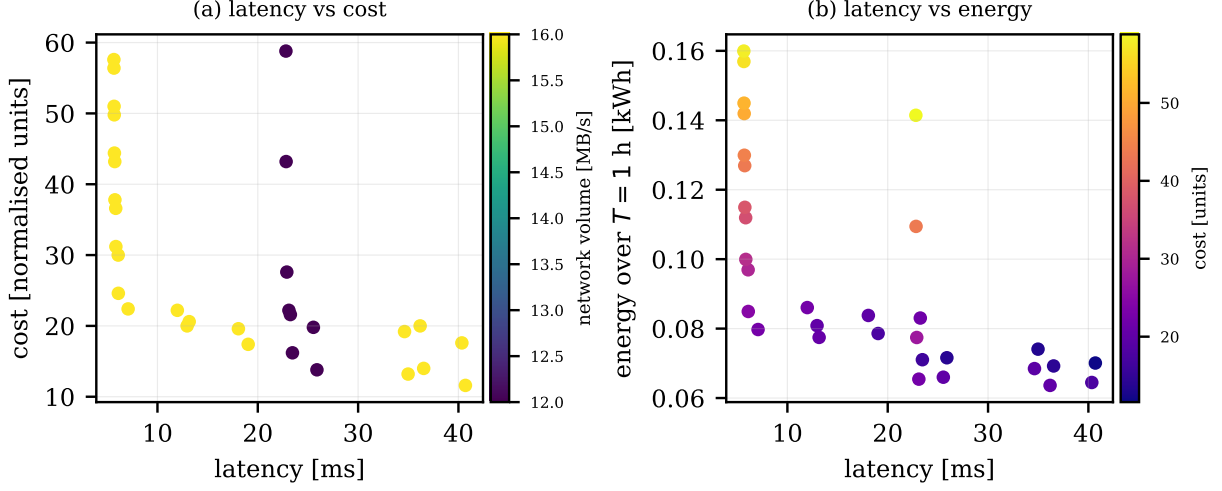


Figure 5: Exact Pareto front recovered on S3/telemetry. (a) latency versus cost, colour encodes network volume; (b) latency versus energy over the accounting horizon $T = 1$ h, colour encodes cost.

6.3. RQ3: when pruning pays off

No positive break-even evaluation cost was observed on either scale: with the closed-form evaluator HCA-DSE is already faster than exhaustive enumeration, so the linear crossover model places the threshold below zero and the projections below are gains that grow with evaluator cost rather than gains that start at one. S2: projected speed-up at the measured DES cost is $77.0\text{--}147.1\times$, with asymptotic evaluation-count ratios of $77.8\text{--}149.5\times$. S3: projected speed-up at the measured DES cost is $589.4\text{--}758.2\times$, with asymptotic evaluation-count ratios of $617.1\text{--}803.7\times$. The DES anchor is 33.6 ms, averaged over 40 designs. The curve in Fig. 6 is a cost projection from measured counts and median overheads; neither exploration strategy was executed end to end with DES in the inner loop. With DES as the leaf evaluator, the returned front would be exact with respect to the DES objective only if the analytical bounds were also admissible for it, which is not established here; the projection quantifies the evaluation budget, not a DES-level guarantee. The measured analytical-evaluator speed-up across S2/S3 is $2.58\text{--}4.96\times$. These hardware- and implementation-specific values do not establish simulator-level Pareto exactness or a universal crossover cost.

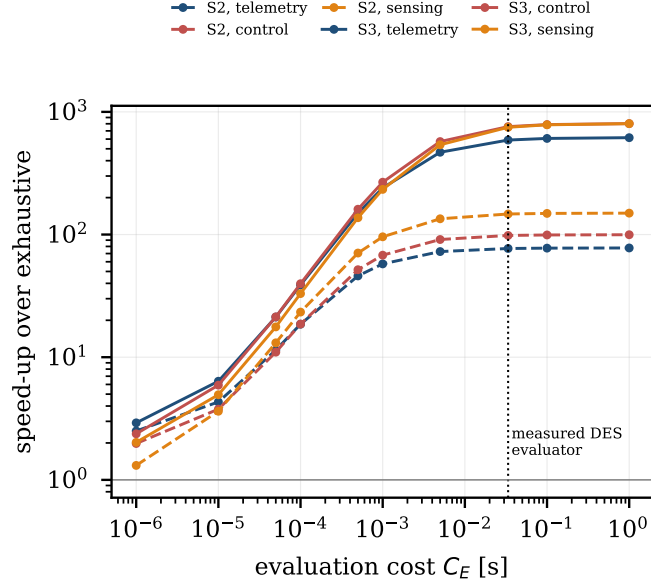


Figure 6: Speed-up over exhaustive enumeration projected as a function of the evaluator cost C_E from the measured overheads and evaluation counts. The dotted line marks the measured cost of the discrete-event evaluator.

6.4. RQ4: comparison with budget-matched exploration

Table 6 reports medians over 20 seeds, with budgets charged to actual full evaluator calls. S2: at $B = 1000$, NSGA-II median recall is 0.900–0.944 and median HV ratio is 0.999–1.000; random-search median recall is 0.160–0.188. S3: at $B = 1000$, NSGA-II median recall is 0.645–0.741 and median HV ratio is 0.969–0.991; random-search median recall is 0.000–0.018. On S3 at $B = 500/1000$, 4 of the six exploratory one-sided comparisons have unadjusted $p < 0.05$; individual unadjusted p -values range from < 0.001 to 0.168 and Cliff's δ from 0.180–0.800. The individual results are in Table 7; no correction for multiple comparisons is applied. HCA-DSE recovers the full analytical front on S3 with 43–56 evaluations. All 960 stochastic runs satisfy $N_E = B$ and proposals $= N_E +$ structural rejections. This accounting includes infeasible full evaluations. Figure 7 shows seed-level HV distributions; Supplementary Tables S1–S3 retain all budgets.

Table 6: Budget-matched comparison. Medians over 20 seeds for the stochastic methods at $B = 1000$; the full sweep is in Supplementary Tables S1–S2.

scale	workload	method	evals	recall	HV ratio	IGD ⁺	time [s]
S2	telemetry	exhaustive	3 888	1.000	1.000	0.0000	0.060
		random ($B=1000$)	1000	0.185	0.923	0.0451	0.017
		NSGA-II ($B=1000$)	1000	0.944	0.999	0.0003	0.184
		HCA-DSE	50	1.000	1.000	0.0000	0.017
S2	control	exhaustive	3 888	1.000	1.000	0.0000	0.050
		random ($B=1000$)	1000	0.160	0.923	0.0379	0.017
		NSGA-II ($B=1000$)	1000	0.900	0.999	0.0007	0.197
		HCA-DSE	39	1.000	1.000	0.0000	0.015
S2	sensing	exhaustive	3 888	1.000	1.000	0.0000	0.035
		random ($B=1000$)	1000	0.188	0.866	0.0622	0.012
		NSGA-II ($B=1000$)	1000	0.938	1.000	0.0009	0.235
		HCA-DSE	26	1.000	1.000	0.0000	0.012
S3	telemetry	exhaustive	34 560	1.000	1.000	0.0000	0.354
		random ($B=1000$)	1000	0.016	0.699	0.1147	0.013
		NSGA-II ($B=1000$)	1000	0.645	0.987	0.0415	0.199
		HCA-DSE	56	1.000	1.000	0.0000	0.063
S3	control	exhaustive	34 560	1.000	1.000	0.0000	0.323
		random ($B=1000$)	1000	0.018	0.691	0.1248	0.012
		NSGA-II ($B=1000$)	1000	0.648	0.991	0.0244	0.209
		HCA-DSE	43	1.000	1.000	0.0000	0.065
S3	sensing	exhaustive	34 560	1.000	1.000	0.0000	0.356
		random ($B=1000$)	1000	0.000	0.600	0.1747	0.012
		NSGA-II ($B=1000$)	1000	0.741	0.969	0.1670	0.218
		HCA-DSE	43	1.000	1.000	0.0000	0.074

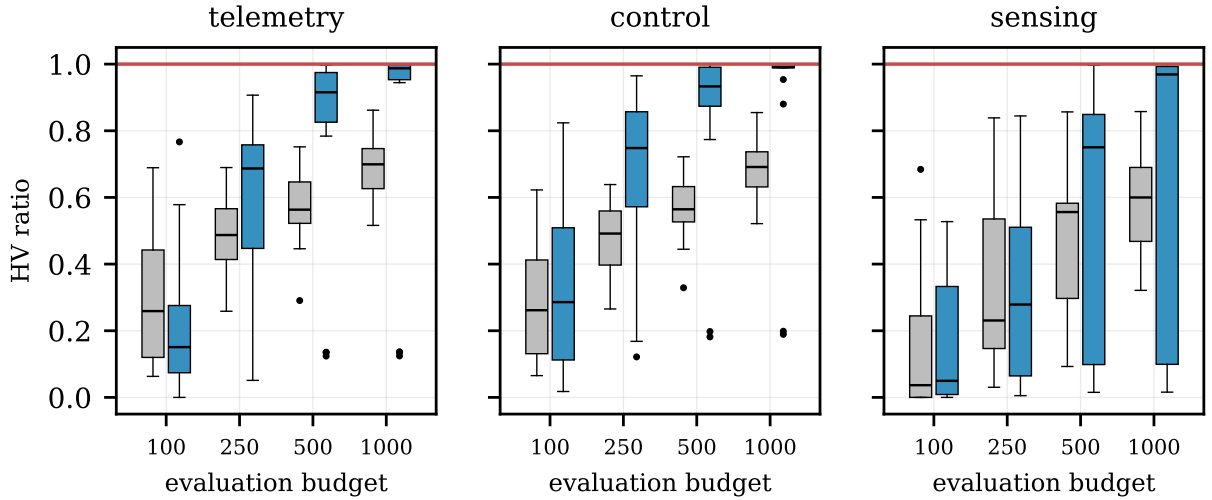


Figure 7: Hypervolume ratio against the exhaustive front on S3 over 20 seeds. Grey: random search; blue: NSGA-II. The red line marks the exact front recovered by HCA-DSE.

Table 7: NSGA-II versus random search on S3 at $B = 500$ and 1000 over 20 seeds: median HV ratio, one-sided Mann–Whitney U p -value and Cliff’s δ . HCA-DSE attains HV ratio 1.0 deterministically with the number of evaluations given in the last column.

workload	budget	NSGA-II HV med.	random HV med.	p	Cliff’s δ	NSGA-II recall	HCA-DSE evals
telemetry	500	0.9154	0.5633	<0.001	0.7	0.355	56
telemetry	1000	0.9874	0.6993	<0.001	0.7	0.645	56
control	500	0.9331	0.5644	<0.001	0.8	0.278	43
control	1000	0.9906	0.6913	<0.001	0.8	0.648	43
sensing	500	0.7504	0.5561	0.168	0.18	0.37	43
sensing	1000	0.9689	0.5998	0.054	0.3	0.741	43

6.5. Ablation: what each layer contributes

The layer ablation on S3 (Table 8 and Fig. 8) separates the contributions of the three pruning layers. Structural pruning alone leaves 34 560 full evaluations. Structural plus feasibility bounds leave 7025–12700, while structural plus dominance bounds leave 328–412. Combining all layers leaves 43–56. Every variant preserves the exhaustive analytical front; the separate layers contribute complementary reductions on these spaces.

Table 8: Ablation on S3. For each workload: expanded partial states, full evaluations and runtime.

variant	telemetry			control			sensing		
	states	evals	[ms]	states	evals	[ms]	states	evals	[ms]
structural only	61 225	34 560	482	61 225	34 560	415	61 225	34 560	404
structural + feasibility bounds	24 650	12 700	613	15 979	7 025	458	23 118	10 956	537
structural + dominance bounds	4 622	328	174	4 730	405	169	4 804	412	153
HCA-DSE (all layers)	2 401	56	88	2 375	43	87	3 221	43	102

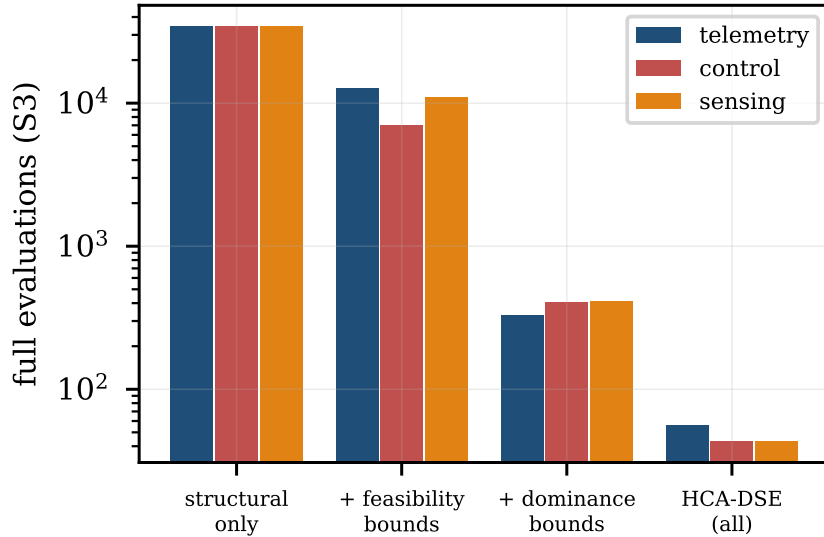


Figure 8: Full evaluations per pruning configuration on S3 (logarithmic scale).

6.6. Variable order and early queueing contributions

Table 9 crosses traversal order and queue-bound activation on S3 (all scales in the supplement). The canonical determinants-early order assigns node counts, node classes, replication and placement policy before the remaining variables; the determinants-late order assigns replication and policy last, so the waiting term stays relaxed to zero on every incomplete design. With the queue bound enabled, the canonical order performs 43–56 full evaluations against 41–54 for the determinants-late order, while expanding 2375–3221 partial designs against 4393–5832. Under the canonical order 3777–5139 checked incomplete designs carry a positive queueing contribution, of which 163–419 are discarded only because that term is present, against the same archive; under the determinants-late order the count is zero by construction. All 36 scale/workload/order/queue combinations preserve the unrounded exhaustive objective-vector set. Changing the order also changes the archive discovery sequence, so only the within-order on/off comparison isolates the bound toggle. On S3, O2 reduces expanded states in each workload, but uses two more full evaluations than O1 for telemetry and control, and the same number for sensing. Within O2, enabling the queue bound reduces full evaluations for all three workloads; its extra computation means that this need not reduce runtime with the cheap analytical evaluator. Thus early queueing pruning is demonstrable, while the best traversal depends on the cost metric.

Table 9: Order/queue ablation on S3. “Active” counts checked incomplete prefixes with positive queue contribution; “pruned” counts those rejected only with that contribution against the same archive. Runtime is the median of three uninstrumented runs.

scale	workload	order	queue	states	checks	evals	active	pruned	ms
S3	telemetry	determinants-early	off	2871	6806	251	0	0	73.3
S3	telemetry	determinants-early	on	2401	6100	56	3777	188	71.8
S3	telemetry	determinants-late	off	6029	14794	251	0	0	169.5
S3	telemetry	determinants-late	on	5832	14794	54	0	0	205.1
S3	control	determinants-early	off	2755	6636	206	0	0	65.9
S3	control	determinants-early	on	2375	6074	43	3777	163	78.6
S3	control	determinants-late	off	5990	14812	205	0	0	155.6
S3	control	determinants-late	on	5826	14812	41	0	0	174.0
S3	sensing	determinants-early	off	4351	9905	537	0	0	107.9
S3	sensing	determinants-early	on	3221	8281	43	5139	419	86.2
S3	sensing	determinants-late	off	4888	11367	538	0	0	103.4
S3	sensing	determinants-late	on	4393	11367	43	0	0	109.4

6.7. Small-space exact solver comparison

Two exact solver baselines bracket the method (Table 10). The finite-table Z3 baseline builds the whole structurally valid relation before solving, so it spends 1080 evaluator calls on S1 and 3888 on S2, with a total runtime of 0.217–4.696 s. A compact factorised SMT encoding avoids evaluator calls altogether: each term of the model is tabulated only over the decisions it depends on (2188–2406 table entries for the 51 840 candidates of S3), objectives and constraints become linear, and the exact Pareto set is enumerated by guided improvement. It recovers the exhaustive front in all nine S1–S3 cases with no full-design evaluation, in 1.4–13.8 s on S1/S2 and 157–170 s on S3, against 5.9–104.1 ms for HCA-DSE with the closed-form evaluator. Solver times depend on the encoding and are indicative. The factorised encoding requires the entire objective and constraint model to be representable by its tables; HCA-DSE requires only admissible bounds on partial designs, and the front it returns is exact for the objective those bounds are admissible for — in this study, the analytical model.

Table 10: Exact finite-table Z3 baseline. Solver time includes encoding; total time also includes full table evaluation. “Calls” counts full evaluator invocations.

scale	workload	method	evaluator calls	tabulated entries	solver checks	total [s]	front
S1	telemetry	HCA-DSE	34	–	–	0.008	17
		finite-table SMT	1080	1080	44	0.452	17
		factorised SMT	0	221	51	2.698	17
S1	control	HCA-DSE	28	–	–	0.008	18
		finite-table SMT	1080	1080	48	0.468	18
		factorised SMT	0	225	52	3.186	18
S1	sensing	HCA-DSE	16	–	–	0.006	11
		finite-table SMT	1080	1080	31	0.217	11
		factorised SMT	0	199	37	1.409	11
S2	telemetry	HCA-DSE	50	–	–	0.020	27
		finite-table SMT	3888	3888	69	4.696	27
		factorised SMT	0	529	76	10.634	27
S2	control	HCA-DSE	39	–	–	0.018	25
		finite-table SMT	3888	3888	71	3.861	25
		factorised SMT	0	537	76	13.812	25
S2	sensing	HCA-DSE	26	–	–	0.015	16
		finite-table SMT	3888	3888	55	1.279	16
		factorised SMT	0	469	54	6.757	16
S3	telemetry	HCA-DSE	56	–	–	0.084	31
		factorised SMT	0	2384	93	170.184	31
S3	control	HCA-DSE	43	–	–	0.088	27
		factorised SMT	0	2406	78	157.413	27
S3	sensing	HCA-DSE	43	–	–	0.104	27
		factorised SMT	0	2188	85	168.624	27

6.8. RQ5: scalability

The scalability sweep (Table 11 and Fig. 9) covers raw spaces from 1.3×10^3 to 8.3×10^7 candidates. Across the sweep, expanded partial states never exceed 18082; full evaluations range from 18–63. The largest observed runtime is 5.52 s, measured with Python memory tracing enabled, and peak traced allocations are 75.3 KiB. This is traced Python memory during search, not process RSS or total interpreter memory.

Table 11: Scalability. For each workload: expanded partial states, full evaluations and runtime [s].

$ \mathcal{X} $	telemetry			control			sensing		
	states	evals	[s]	states	evals	[s]	states	evals	[s]
1.3×10^3	290	46	0.09	268	34	0.08	234	18	0.05
5.8×10^3	701	59	0.23	682	54	0.21	649	30	0.16
5.2×10^4	2 751	62	1.00	2 732	57	0.86	3 495	53	1.08
3.5×10^5	5 097	63	1.95	5 074	58	1.71	8 228	59	2.26
2.8×10^6	8 030	63	2.40	8 007	58	2.26	13 986	61	3.97
1.6×10^7	9 257	63	2.92	9 234	58	2.73	16 345	61	5.17
8.3×10^7	10 338	63	3.07	10 315	58	3.56	18 082	61	5.52

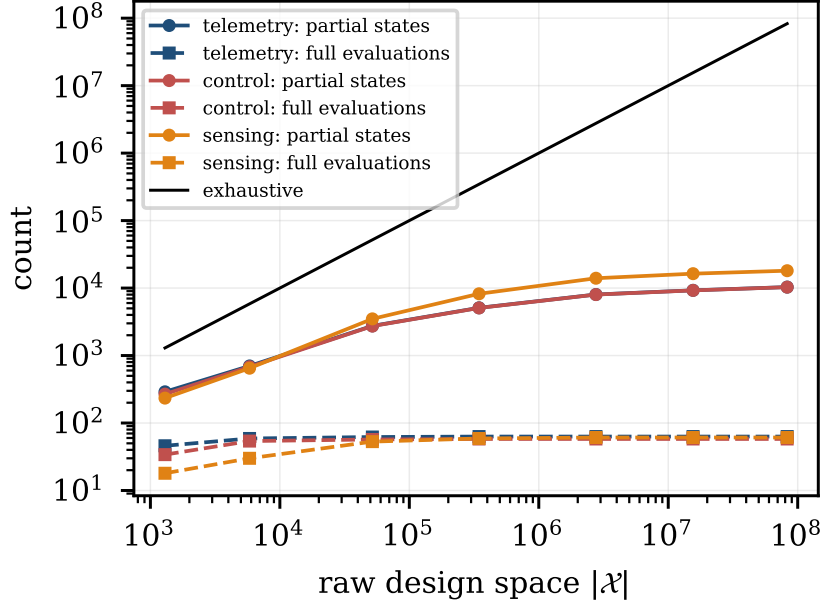


Figure 9: Expanded partial states and full evaluations against the raw design-space size.

6.9. RQ6: transfer to a real deployment

Table 12 reports analytical mean-latency predictions against the revised DES model. The independent Docker round retains the same 36 configurations and all 108 runs with the analytical model frozen. Table 13 compares predictions with the median of three measured run means. Pooled MAPE is 25.4% and median absolute error is 6.51 ms. Within-workload pair agreement above 25% is 88.6–91.3%, compared with 97.8% over 498 pooled pairs. Pooled agreement benefits from separation between workloads and is not a substitute for within-workload validation. 6 configurations have a ratio of largest to smallest repeated mean latency of at least 1.5; all remain in the main summary. Supplementary Table S4 separately reports the p_{50} comparison and its utilisation/repeat subsets. Differences between predicted means and measured medians must not be interpreted as direct mean-prediction error. Agreement grows with the separation the model predicts (Fig. 10b): 86% of all 196 within-workload pairs are ordered as predicted (95% architecture-level bootstrap interval 77–94%), 90% of the 105 pairs the model separates by more than 25% (77–98%), 53 of 54 pairs separated by more than 50% and 20 of 20 separated by more than 100%.

Table 12: Analytical mean latency versus DES: 40 distinct designs per workload, five seeds per design. Simulated latency is the median of the five run means.

workload	designs	MAPE [%]	Spearman ρ
telemetry	40	6.9	0.986
control	40	6.1	0.988
sensing	40	20.5	0.955

Table 13: Independent testbed validation against measured run means. Pair agreement is the share of architecture pairs whose measured order matches the model order, among pairs the model separates by more than 25%. ALL pools workloads. p_{50} subsets and 10% results appear in Supplementary Table S4.

workload	n	MAPE [%]	Spearman ρ	agreement > 25%	pairs
telemetry	12	17.8	0.776	0.895	38
control	12	12.0	0.865	0.886	44
sensing	12	46.3	0.921	0.913	23
ALL	36	25.4	0.967	0.978	498

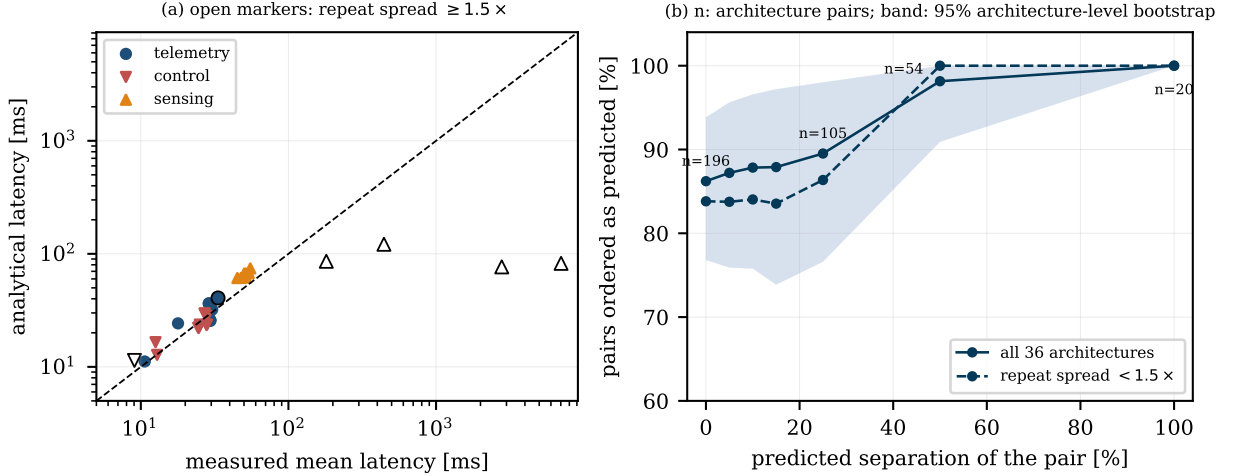


Figure 10: (a) Analytical latency against the median of three measured mean latencies on the independent containerised testbed round. Open markers denote architectures whose repeated mean latencies differ by a factor of 1.5 or more. (b) Share of within-workload architecture pairs whose measured order matches the predicted order, as a function of the predicted relative separation; band: 95% architecture-level cluster-bootstrap interval.

6.10. Independent validation after model correction

Earlier deployment measurements motivated replacing an M/M/1-inspired waiting term with the M/D/1-inspired expression. This historical change was post hoc; the independent round reported here uses a frozen model and is not fitted to its observations. Deterministic service motivates the factor of two only in an idealised single-server queue. It does not establish an exact queueing formula for the full deployment, so we retain the measured errors and unstable repeats as limitations rather than adjusting the model after validation.

7. Discussion

7.1. Why hierarchical pruning works here

The ablation shows that structural restrictions, feasibility bounds and an incumbent Pareto archive provide complementary reductions on these design spaces. The default search does not need early queueing contributions to obtain the reported evaluation savings. The separate order experiment (Section 6.6) measures the effect of assigning their determinants earlier; it also makes clear that changing order changes which incumbents become available for dominance pruning.

7.2. When HCA-DSE is the right tool — and when it is not

The method is advantageous when the design space is large, the constraints are tight, the evaluator is expensive, and admissible bounds can be constructed. The crossover analysis quantifies the third condition using actual exhaustive evaluator calls and measured overhead differences. At the measured DES cost it projects up to $758\times$ speed-up, but does not execute DES-based search or establish its exactness. Industrial evaluation costs can be substantially higher [1]; transfer of the numerical speed-ups nevertheless requires new measurements and bounds appropriate to that evaluator.

Conversely, the method offers little when the space is small, the constraints are loose, the evaluation is nearly free, or — most importantly — when no admissible bound can be constructed for the objectives of interest. If a bound cannot be proven admissible it must not be used: the guarantee of Theorem 1 rests entirely on that property, and an inadmissible bound silently converts an exact method into an approximate one without any indication in the output.

7.3. Relation to symbolic exact exploration

Symbolic approaches [4] prune partial designs with a constraint solver and are, in principle, more general than closed-form bounds: any constraint the solver can express can participate in the pruning. The assignment-monotone construction of the cited method does not directly supply our queueing bounds. Our construction instead uses objective-specific relaxations. These bounds preserve the analytical front, although the default order activates the exact queueing term only at full assignment. The order ablation tests earlier activation. Our finite-table Z3 baseline is exact but builds all evaluator values first; a compact solver front end with partial-design propagation remains an untested alternative.

7.4. Relation to evaluator-centric methodologies

CompDSE [1] and HCA-DSE answer different questions. The former asks how a design point of an industrial dCPS can be modelled and evaluated faithfully; the latter asks which design points need not be evaluated at all. Their combination is conditional: analytical bounds would need to be proved admissible for the trace-derived simulator itself. Admissibility for the analytical model and good empirical rank agreement do not establish that condition. The crossover analysis is a cost projection and supplies no simulator-level exactness guarantee.

7.5. Accuracy versus ordering

The independent validation assesses both prediction error and ordering. Within-workload agreement with measured mean-latency order is 88.6–91.3% on pairs separated by more than 25% in predicted latency; the pooled value is 97.8%. Pooling can benefit from workload separation and should not be read as evidence that every local design choice is reliable. Repeated runs also expose instability that a fixed analytical objective cannot represent.

The theorem guarantees the Pareto front of the analytical model. It does not imply that this front equals the deployment or DES front. Analytical exploration can select candidates for higher-fidelity reassessment, but discarding all other candidates preserves that evaluator’s front only if the pruning bounds are also proved admissible for it. The measured-mean and measured- p_{50} comparisons are reported separately because an expected-latency approximation does not directly predict a latency quantile.

8. Threats to validity

Internal validity. The correctness of the method rests on the admissibility of the bounds. We address this at three levels: proofs (Lemmas 1–2, Propositions 1–3, Theorem 1); an exhaustive machine check of $LB_i(p) \leq f_i(x)$ over all prefixes and all completions on S1 for the three workloads; and a direct check that, for every pruned prefix, no completion contradicts the claim of the corresponding rule. We deliberately separate *theoretical* correctness from *implementation* correctness and verify both, since an admissible bound that is implemented incorrectly is indistinguishable, in the output, from an inadmissible one. Structural pruning is part of the definition of the design space rather than a consequence of the constraints, and the exhaustive baseline uses the same predicate; this was discovered by the pruning-safety test and is stated explicitly rather than absorbed into the comparison.

Post-hoc model correction. Earlier measurements motivated the M/D/1-inspired waiting correction. The independent second round freezes this model and all 36 configurations beforehand, so its results are not in-sample calibration scores. Selection of these configurations still reflects the original study, and a single-machine repeat is not an independent deployment population. The idealised queueing argument does not determine the sign of deployment error.

Construct validity. The analytical model uses per-node service times with a stage-wise M/D/1-inspired waiting multiplier based on average tier utilisation. It is neither an exact aggregate-server queue nor an exact multi-server queue, and its error has no proved sign. Transport protocol effects are ignored, the power model is linear in utilisation, and costs are normalised units. Testbed validation quantifies latency error and ordering agreement, but does not prove admissibility for a simulator or for the deployment. The model omits transport-level and host-scheduling effects, which can be especially consequential for large payloads and near saturation. The energy and cost models are not validated against measurements at all; they are used for comparative exploration only, and no absolute energy or cost claim is made.

External validity. The workloads are synthetic, although chosen to be qualitatively different rather than rescaled versions of one another, and the architecture template is fixed to four tiers with three stages. The empirical study is further restricted to configurations that fit on a single machine (≤ 4 edge nodes, ≤ 2 gateways), and link shaping is applied in the forward direction of each hop, matching the one-way transfers counted by the model. Results on a geographically distributed deployment may differ, particularly for the WAN hop.

Conclusion validity. Search counts are deterministic for fixed inputs and order; wall-clock timings still vary. Stochastic baselines use 20 seeds and exploratory, unadjusted Mann–Whitney tests with Cliff’s δ . Testbed configurations have only three repeats. We report the median of run means, repeat IQR and a largest/smallest ratio; unstable configurations remain in the main results. This sample does not characterise tail-risk probabilities.

Evaluation-budget accounting. All stochastic runs stop at exactly B full evaluator calls, with structurally rejected proposals counted separately. Calls returning infeasibility consume budget, as do repeat evaluations. Thus budgets match full invocations rather than unique feasible designs or wall-clock work. NSGA-II duplicate elimination and random sampling with replacement remain distinct search choices, not equal unique-design budgets.

Limitations. The method assumes a static workload during one exploration, does not model run-time migration or autoscaling, and requires bounds to be derived per objective — a manual step that must be repeated when the objective set changes. The finite-table Z3 baseline on S1/S2 is an exact encoding, but fully evaluates the space before solving. It does not establish superiority over compact SMT, CP-SAT or mixed-integer formulations with native propagation. Exact integer allocation is practical for related problems [17, 18]; a comparable compact encoding of this architecture model remains future work. These limits constrain claims of general algorithmic efficiency beyond the measured competitors.

9. Conclusions

In the class of heterogeneous dCPS design spaces studied here, exploration cost is strongly driven by full design-point evaluation cost as well as combinatorial space size. HCA-DSE combines structural, feasibility and dominance pruning with objective-specific admissible relaxations for a finite edge-cloud architecture model. The construction preserves the exact analytical Pareto front; safe pruning and evaluator avoidance are established principles, while the contribution here is their concrete bound construction and experimental assessment for this model.

The default method performs 16–63 full evaluations across the reported spaces, including a sweep up to 8.3×10^7 raw candidates. Exhaustive reference comparisons on S1–S3 recover identical unrounded objective-vector sets. Stochastic baselines now spend precisely their stated full-call budgets; at $B = 1000$, NSGA-II median recall ranges from 0.65–0.94. The cost model projects up to $758\times$ speed-up using the measured 33.6 ms DES cost, without claiming an end-to-end DES search measurement. The order/queue ablation distinguishes early bound activation from traversal effects, and the finite-table SMT comparison provides a small-space exact reference whose table-construction cost is explicitly included.

The independent Docker round uses 36 fixed architectures and 108 runs without refitting the model. Agreement with measured mean-latency order is 88.6–91.3% within workloads for pairs separated by more than 25% in the model, compared with 97.8% pooled. These measurements assess transfer of analytical rankings; they neither validate energy/cost objectives nor establish deployment-level Pareto exactness. Further work should develop compact symbolic solver encodings and tighter bounds proved admissible for higher-fidelity evaluators.

Data availability

The complete implementation, all experiment scripts, the testbed and every result file required to regenerate the figures and tables of this paper are available in the replication package accompanying the submission.

References

- [1] F. S. Saadatmand, T. Stefanov, I. González Alonso, A. D. Pimentel, B. Akesson, CompDSE: A methodology for design space exploration of computing subsystems within complex cyber-physical systems, *IET Cyber-Physical Systems: Theory & Applications* 10 (1) (2025) e70019. doi:10.1049/cps2.70019.
- [2] M. Herget, F. S. Saadatmand, M. Bor, I. González Alonso, T. Stefanov, B. Akesson, A. D. Pimentel, Design space exploration for distributed cyber-physical systems: State-of-the-art, challenges, and directions, in: 25th Euromicro Conference on Digital System Design (DSD), 2022, pp. 632–640. doi:10.1109/DSD57027.2022.00090.

- [3] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: NSGA-II, *IEEE Transactions on Evolutionary Computation* 6 (2) (2002) 182–197. doi:10.1109/4235.996017.
- [4] K. Neubauer, P. Wanko, T. Schaub, C. Haubelt, Exact multi-objective design space exploration using ASPmT, in: *Design, Automation & Test in Europe Conference (DATE)*, 2018, pp. 257–260. doi:10.23919/DATE.2018.8342014.
- [5] C. Haubelt, K. Neubauer, T. Schaub, P. Wanko, Design space exploration with answer set programming, *KI - Künstliche Intelligenz* 32 (2018) 205–206. doi:10.1007/s13218-018-0530-3.
- [6] S. Petchrompo, D. W. Coit, A. Brintrup, A. Wannakrairot, A. K. Parlikad, A review of Pareto pruning methods for multi-objective optimization, *Computers & Industrial Engineering* 167 (2022) 108022. doi:10.1016/j.cie.2022.108022.
- [7] L. Mandow, J. L. Pérez-de-la Cruz, Multiobjective A* search with consistent heuristics, *Journal of the ACM* 57 (5) (2010) 1–25. doi:10.1145/1754399.1754400.
- [8] K. Neubauer, B. Beichler, C. Haubelt, Exact design space exploration based on consistent approximations, *Electronics* 9 (7) (2020) 1057. doi:10.3390/electronics9071057.
- [9] B. S. Stewart, C. C. White, Multiobjective A*, *Journal of the ACM* 38 (4) (1991) 775–814. doi:10.1145/115234.115368.
- [10] B. Kienhuis, E. F. Deprettere, P. van der Wolf, K. Vissers, A methodology to design programmable embedded systems: The Y-chart approach, in: *Embedded Processor Design Challenges*, Springer, 2002, pp. 18–37. doi:10.1007/3-540-45874-3_2.
- [11] A. Varga, R. Hornig, An overview of the OMNeT++ simulation environment, in: *1st International Conference on Simulation Tools and Techniques for Communications, Networks and Systems (SIMUTools)*, 2008, pp. 1–10. doi:10.4108/ICST.SIMUTOOLS2008.3027.
- [12] Y. Xiao, C. Oh, M. Lora, P. Nuzzo, Efficient exploration of cyber-physical system architectures using contracts and subgraph isomorphism, in: *Design, Automation & Test in Europe Conference (DATE)*, 2024, pp. 1–6. doi:10.23919/DATE58400.2024.10546764.
- [13] M. Amir, T. Givargis, Pareto optimal design space exploration of cyber-physical systems, *Internet of Things* 12 (2020) 100308. doi:10.1016/j.iot.2020.100308.
- [14] F. S. Saadatmand, T. Stefanov, I. G. Alonso, A. D. Pimentel, B. Akesson, M. Herget, M. Bor, Automated Derivation of Application Workload Models for Design Space Exploration of Industrial Distributed Cyber-Physical Systems, in: *2024 IEEE 7th International Conference on Industrial Cyber-Physical Systems (ICPS)*, 2024, pp. 1–8. doi:10.1109/icps59941.2024.10639941.
- [15] X. Guo, A. D. Pimentel, T. Stefanov, Automated Exploration and Implementation of Distributed CNN Inference at the Edge, *IEEE Internet of Things Journal* 10 (7) (2023) 5843–5858. doi:10.1109/jiot.2023.3237572.
- [16] X. Guo, Q. Jiang, A. D. Pimentel, T. Stefanov, Model and system robustness in distributed CNN inference at the edge, *Integration* 100 (2025) 102299. doi:10.1016/j.vlsi.2024.102299.
- [17] A. Kouloumpis, G. L. Stavrinos, M. K. Michael, T. Theocharides, An optimization framework for task allocation in the edge/hub/cloud paradigm, *Future Generation Computer Systems* 155 (2024) 354–366. doi:10.1016/j.future.2024.02.005.
- [18] A. Kouloumpis, G. L. Stavrinos, M. K. Michael, T. Theocharides, A reliability- and latency-driven task allocation framework for workflow applications in the edge–hub–cloud continuum, *Future Generation Computer Systems* 180 (2026) 108414. doi:10.1016/j.future.2026.108414.
- [19] H. Sedghani, F. Filippini, D. Ardagna, SPACE4AI-D: A design-time tool for AI applications resource selection in computing continua, *IEEE Transactions on Services Computing* 17 (6) (2024) 4324–4339. doi:10.1109/TSC.2024.3479935.
- [20] D. Casini, P. Pazzaglia, M. Becker, Managing real-time constraints through monitoring and analysis-driven edge orchestration, *Journal of Systems Architecture* 163 (2025) 103403. doi:10.1016/j.sysarc.2025.103403.
- [21] L. Ma, Y. Zhang, J. Zhou, G. Zhang, A gene-inspired metaheuristic for scheduling workflow tasks in mobile edge computing-supported cyber-physical systems, *Journal of Systems Architecture* 151 (2024) 103136. doi:10.1016/j.sysarc.2024.103136.
- [22] M. N. Ali, I. Ullah, M. Imran, M. Salah ud din, B.-S. Kim, Dependency-aware microservices offloading in ICN-based edge computing testbed, *Journal of Systems Architecture* 171 (2026) 103663. doi:10.1016/j.sysarc.2025.103663.
- [23] J. Wang, M. Zhang, H. Duan, K. Mei, D. Wang, M. Qiu, A co-optimization framework toward energy-efficient cloud-edge inference with stochastic computing and precision-compensating NAS, *Journal of Systems Architecture* 172 (2026) 103693. doi:10.1016/j.sysarc.2026.103693.
- [24] F. Filippini, M. Lujak, M. Ciavotta, Distributed replica allocation and load balancing for Edge-Cloud FaaS,

- Journal of Systems Architecture 176 (2026) 103787. doi:10.1016/j.sysarc.2026.103787.
- [25] P. Rodríguez, J. Mateos-Bravo, S. Laso, J. L. Herrera, J. Berrocal, Predictively controlling the computing continuum with distributed energy-aware orchestration, *Journal of Systems Architecture* 175 (2026) 103752. doi:10.1016/j.sysarc.2026.103752.
 - [26] J. Blank, K. Deb, pymoo: Multi-objective optimization in Python, *IEEE Access* 8 (2020) 89497–89509. doi:10.1109/ACCESS.2020.2990567.
 - [27] E. Zitzler, L. Thiele, Multiobjective evolutionary algorithms: a comparative case study and the strength Pareto approach, *IEEE Transactions on Evolutionary Computation* 3 (4) (1999) 257–271. doi:10.1109/4235.797969.
 - [28] H. Ishibuchi, H. Masuda, Y. Tanigaki, Y. Nojima, Modified distance calculation in generational distance and inverted generational distance, in: *Evolutionary Multi-Criterion Optimization*, Springer, 2015, pp. 110–125. doi:10.1007/978-3-319-15892-1_8.
 - [29] C. Audet, J. Bignon, D. Cartier, S. Le Digabel, L. Salomon, Performance indicators in multiobjective optimization, *European Journal of Operational Research* 292 (2) (2021) 397–422. doi:10.1016/j.ejor.2020.11.016.
 - [30] T. Blickle, J. Teich, L. Thiele, System-level synthesis using evolutionary algorithms, *Design Automation for Embedded Systems* 3 (1) (1998) 23–58. doi:10.1023/A:1008899229802.
 - [31] C. Haubelt, J. Teich, Accelerating design space exploration using Pareto-front arithmetics, in: *Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2003, pp. 525–531. doi:10.1145/1119772.1119882.
 - [32] M. Lukasiewicz, M. Glaß, C. Haubelt, J. Teich, Efficient symbolic multi-objective design space exploration, in: *Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2008, pp. 691–696. doi:10.1109/ASPDAC.2008.4484040.
 - [33] L. Kleinrock, *Queueing Systems, Volume 1: Theory*, Wiley, New York, 1975.
 - [34] M. Harchol-Balter, *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*, Cambridge University Press, 2013. doi:10.1017/CB09781139226424.
 - [35] H. B. Mann, D. R. Whitney, On a test of whether one of two random variables is stochastically larger than the other, *The Annals of Mathematical Statistics* 18 (1) (1947) 50–60. doi:10.1214/aoms/1177730491.
 - [36] N. Cliff, Dominance statistics: Ordinal analyses to answer ordinal questions, *Psychological Bulletin* 114 (3) (1993) 494–509. doi:10.1037/0033-2909.114.3.494.
 - [37] L. de Moura, N. Bjørner, Z3: An Efficient SMT Solver, in: *Tools and Algorithms for the Construction and Analysis of Systems*, Vol. 4963, 2008, pp. 337–340. doi:10.1007/978-3-540-78800-3_24.
 - [38] S. Hemminger, Network emulation with NetEm, in: *Linux Conference Australia (linux.conf.au)*, 2005.